

Time Series Forecasting — Versi 1: Klasik & Machine Learning

Saham: UNTR.JK | **Data:** 10 Tahun | **Test Set:** 1 Tahun Terakhir

Model yang digunakan:

#	Model	Kategori
1	Moving Average	Statistik
2	ARIMA	Statistik
3	SARIMA	Statistik
4	SARIMAX	Statistik + Exogenous
5	Prophet	ML/Statistik
6	XGBoost	Machine Learning
7	LSTM	Deep Learning

```
# Install dependencies (otomatis jika di Google Colab)
import subprocess, sys
IN_COLAB = 'google.colab' in sys.modules
if IN_COLAB:
    print("Running di Google Colab – menginstall dependencies...")
    subprocess.run([sys.executable, '-m', 'pip', 'install', 'yfinance', 'prophet',
                    'statsmodels', 'xgboost', '-q'], check=True)
    print("Install selesai!")
else:
    print("Running secara lokal – pastikan requirements_v1.txt sudah diinstall")
```

Running di Google Colab – menginstall dependencies...
Install selesai!

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import seaborn as sns
import warnings, time
warnings.filterwarnings('ignore')
from datetime import datetime, timedelta
import yfinance as yf
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.statespace.sarimax import SARIMAX
from statsmodels.tsa.stattools import adfuller
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from prophet import Prophet
from xgboost import XGBRegressor
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error
```

```

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping

try:
    plt.style.use('seaborn-v0_8-whitegrid')
except:
    plt.style.use('ggplot')

TICKER = 'UNTR.JK'
COLORS = {'actual': '#2196F3', 'MA': '#FF9800', 'ARIMA': '#4CAF50', 'SARIMA': '#F44336',
          'SARIMAX': '#9C27B0', 'Prophet': '#795548', 'XGBoost': '#E91E63', 'LSTM': '#FFEB3B'}
print(f"TensorFlow: {tf.__version__} |  Setup OK")

```

TensorFlow: 2.20.0 |  Setup OK

✓ 1. Data Acquisition

```

end = datetime.today()
start = end - timedelta(days=365*10)
print(f"Mengunduh {TICKER}: {start.date()} → {end.date()}")
raw = yf.download(TICKER, start=start, end=end, auto_adjust=True, progress=F)
df = raw[['Close', 'Volume']].copy().dropna()
df.index = pd.DatetimeIndex(df.index)
print(f"Total: {len(df)} hari trading")
df.tail()

```

Mengunduh UNTR.JK: 2016-06-10 → 2026-06-08
Total: 2457 hari trading

Price	Close	Volume	
Ticker	UNTR.JK	UNTR.JK	
Date			
2026-06-02	22500.0	8257600	
2026-06-03	22000.0	7258900	
2026-06-04	22000.0	6039600	
2026-06-05	21250.0	5219300	
2026-06-08	20875.0	5393000	

✓ 2. Exploratory Data Analysis (EDA)

```

close = df['Close'].squeeze()
volume = df['Volume'].squeeze()

```

```
rm = close.rolling(30).mean()
rs = close.rolling(30).std()

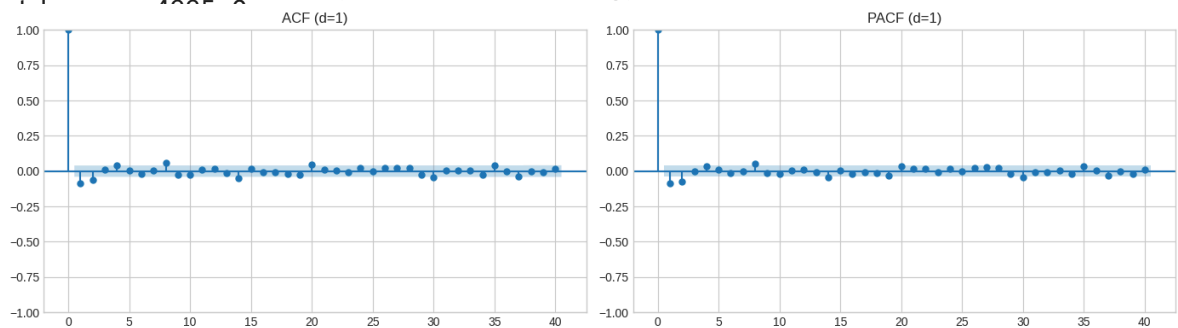
fig, axes = plt.subplots(3, 1, figsize=(14, 12))
fig.suptitle(f'EDA – {TICKER} (10 Tahun Terakhir)', fontsize=14, fontweight=
axes[0].plot(close.index, close, color=COLORS['actual'], lw=1.5); axes[0].se
axes[1].bar(volume.index, volume, color='#4CAF50', alpha=0.6, width=1); axes
axes[2].plot(close.index, close, alpha=0.4, color=COLORS['actual'], lw=1)
axes[2].plot(rm.index, rm, color='#F44336', lw=2, label='MA-30')
axes[2].fill_between(close.index, rm-2*rs, rm+2*rs, alpha=0.15, color='gray'
axes[2].set_title('Rolling Mean & Std (30d)'); axes[2].legend(); axes[2].gri
plt.tight_layout(); plt.savefig('eda_untr.png', dpi=150, bbox_inches='tight'
print(close.describe().round(0))
```




```
adf = adfuller(close.dropna())
print(f"ADF Statistic: {adf[0]:.4f} | p-value: {adf[1]:.4f}")
print(f"→ {'✅ Stasioner' if adf[1]<0.05 else '❌ Tidak Stasioner (perlu di
```

```
fig, axes = plt.subplots(1, 2, figsize=(14, 4))
plot_acf(close.diff().dropna(), lags=40, ax=axes[0]); axes[0].set_title('ACF
plot_pacf(close.diff().dropna(), lags=40, ax=axes[1], method='ywm'); axes[1]
plt.tight_layout(); plt.savefig('acf_pacf.png', dpi=150, bbox_inches='tight'
```

count 2457.0
 ADF Statistic: -2.1300 | p-value: 0.2326
 mean 16580.0
 → ❌ Tidak Stasioner (perlu differencing)



3. Train / Test Split

```

# Test period: 30 trading days (standar portfolio DS - cukup untuk semua mod
N_TEST_DAYS = 30
split_date = df.index[-N_TEST_DAYS]
train_close = close[:split_date]; test_close = close[split_date:]
train_vol = volume[:split_date]; test_vol = volume[split_date:]
n_test = len(test_close)
print(f"Train: {train_close.index[0].date()} -> {train_close.index[-1].date()}")
print(f"Test : {test_close.index[0].date()} -> {test_close.index[-1].date()}")

results, preds = [], {}

def evaluate(y_true, y_pred, name):
    y_true = np.array(y_true).flatten(); y_pred = np.array(y_pred).flatten()
    mask = ~np.isnan(y_true) & ~np.isnan(y_pred); y_true, y_pred = y_true[mask], y_pred[mask]
    mae = mean_absolute_error(y_true, y_pred)
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    mape = np.mean(np.abs((y_true - y_pred) / y_true)) * 100
    print(f"\n{'-'*35}\nModel : {name}\nMAE : {mae:,.2f}\nRMSE : {rmse:,.2f}\nMAPE : {mape:,.2f}")
    results.append({'Model':name, 'MAE':round(mae,2), 'RMSE':round(rmse,2), 'MAPE':round(mape,2)})
    preds[name] = y_pred; return mae, rmse, mape

def plot_forecast(name, pred, color):
    fig, axes = plt.subplots(2, 1, figsize=(14, 10))
    fig.suptitle(f'{name} - Prediksi vs Aktual', fontsize=13, fontweight='bold')

    # Panel atas: seluruh data training + test
    axes[0].plot(train_close.index, train_close.values, color='gray', lw=1, ls='solid')
    axes[0].plot(test_close.index, test_close.values, color=COLORS['actual'], lw=1, ls='solid')
    axes[0].plot(test_close.index[:len(pred)], pred, color=color, lw=2, ls='solid')
    axes[0].axvline(test_close.index[0], color='red', ls=':', lw=1.5, alpha=0.5)
    axes[0].set_title('Overview Keseluruhan (10 Tahun)')
    axes[0].legend(loc='upper left'); axes[0].grid(alpha=0.3); axes[0].set_y

    # Panel bawah: zoom ke periode test saja
    axes[1].plot(train_close.index[-60:], train_close.values[-60:], color='gray', lw=1, ls='solid')
    axes[1].plot(test_close.index, test_close.values, color=COLORS['actual'], lw=1, ls='solid')
    axes[1].plot(test_close.index[:len(pred)], pred, color=color, lw=2, ls='solid')
    axes[1].set_title(f'Zoom: Periode Test ({n_test} Hari Trading)')
    axes[1].legend(loc='upper left'); axes[1].grid(alpha=0.3); axes[1].set_y

    plt.tight_layout()
    plt.savefig(f'pred_{name[:6].lower().replace(" ", "_")}.png', dpi=150, bbbox=[0, 0, 1, 1])
    plt.show()

```

```

Train: 2016-06-10 -> 2026-04-24 (2428 hari)
Test : 2026-04-24 -> 2026-06-08 (30 hari trading = ~6 minggu)

```

✓ 4. Model 1 — Moving Average

```

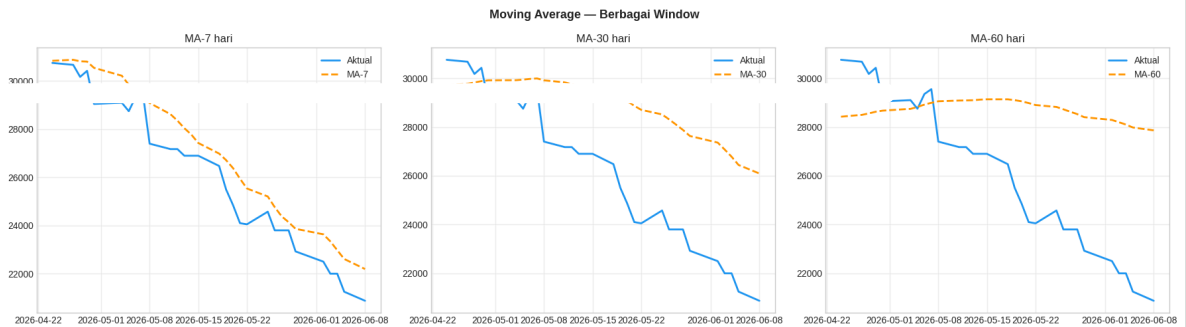
windows = [7, 30, 60]
fig, axes = plt.subplots(1, 3, figsize=(18,5))
fig.suptitle('Moving Average - Berbagai Window', fontsize=13, fontweight='bold')

```

```

for i, w in enumerate(windows):
    ma = close.rolling(w).mean().reindex(test_close.index)
    axes[i].plot(test_close.index, test_close.values, color=COLORS['actual'])
    axes[i].plot(test_close.index, ma.values, color=COLORS['MA'], lw=2, ls='
    axes[i].set_title(f'MA-{w} hari'); axes[i].legend(); axes[i].grid(alpha=
plt.tight_layout(); plt.savefig('pred_ma.png', dpi=150, bbox_inches='tight')
ma30 = close.rolling(30).mean().reindex(test_close.index)
evaluate(test_close.values, ma30.values, 'Moving Average (30d)')

```



```

Model : Moving Average (30d)
MAE   : 2,977.14
RMSE  : 3,417.02
MAPE  : 12.29%
(2977.1368598090276,
 np.float64(3417.0248752458683),
 np.float64(12.289802650359258))

```

5. Model 2 — ARIMA

```

# Walk-Forward: prediksi 1 hari, update dengan aktual, maju
# Sama seperti XGBoost/LSTM yang juga menggunakan lag dari data aktual
print("Fitting ARIMA(5,1,0) + Walk-Forward (1-step rolling)...")
t0 = time.time()
arima_base = ARIMA(train_close, order=(5,1,0)).fit()

```

```
preds_arima, current = [], arima_base
for i in range(n_test):
    preds_arima.append(float(current.forecast(steps=1).iloc[0]))
    current = current.extend(endog=[test_close.values[i]])
arima_fc = pd.Series(preds_arima, index=test_close.index)
print(f"Selesai: {time.time()-t0:.1f}s")
plot_forecast('ARIMA', arima_fc.values, COLORS['ARIMA'])
evaluate(test_close.values, arima_fc.values, 'ARIMA(5,1,0)')
```



```
Fitting ARIMA(5,1,0) + Walk-Forward (1-step rolling)...
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:4:
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:4:
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:4:
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:8:
return get_prediction_index(
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:8:
return get_prediction_index(
Selesai: 1.4s
```



6. Model 3 — SARIMA

```
print("Fitting SARIMA(1,1,1)(1,1,0,5) + Walk-Forward...")
print("(estimasi 2-5 menit)")
t0 = time.time()
sarima_base = SARIMAX(train_close, order=(1,1,1), seasonal_order=(1,1,0,5),
                      enforce_stationarity=False, enforce_invertibility=False)
preds_sarima, current = [], sarima_base
for i in range(n_test):
    preds_sarima.append(float(current.forecast(steps=1).iloc[0]))
    current = current.extend(endog=[test_close.values[i]])
sarima_fc = pd.Series(preds_sarima, index=test_close.index)
print(f"Selesai: {time.time()-t0:.1f}s")
plot_forecast('SARIMA', sarima_fc.values, COLORS['SARIMA'])
evaluate(test_close.values, sarima_fc.values, 'SARIMA(1,1,1)(1,1,0,5)')
```

```
MAE    : 484.06
RMSE   : 659.53
MAPE   : 1.90%
(484.0595259281687,
 np.float64(659.5286581784258),
 np.float64(1.8953142185780592))
```


Fitting SARIMA(1,1,1)(1,1,0,5) + Walk-Forward...

(estimasi 2-5 menit)

```
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:4:
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:4:
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:8:
return get_prediction_index(
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:8:
return get_prediction_index(
```

Selesai: 1.9s



7. Model 4 — SARIMAX (dengan Volume)

```
print("Fitting SARIMAX (Volume) + Walk-Forward...")
```

```
t0 = time.time()
```

```
sarimax_base = SARIMAX(train_close, exog=train_vol.values.reshape(-1,1),
                        order=(1,1,1), seasonal_order=(1,0,0,5),
                        enforce_stationarity=False, enforce_invertibility=False)
```

```
preds_sarimax, current = [], sarimax_base
```

```
for i in range(n_test):
```

```
    fc_s = current.forecast(steps=1, exog=[[test_vol.values[i]]])
```

```
    preds_sarimax.append(float(fc_s.iloc[0]))
```

```
    current = current.extend(endog=[test_close.values[i]], exog=[[test_vol.v
```

```
sarimax_fc = pd.Series(preds_sarimax, index=test_close.index)
```

```
print(f"Selesai: {time.time()-t0:.1f}s")
```

```
plot_forecast('SARIMAX', sarimax_fc.values, COLORS['SARIMAX'])
```

```
evaluate(test_close.values, sarimax_fc.values, 'SARIMAX(Volume)')
```

RMSE : 741.95

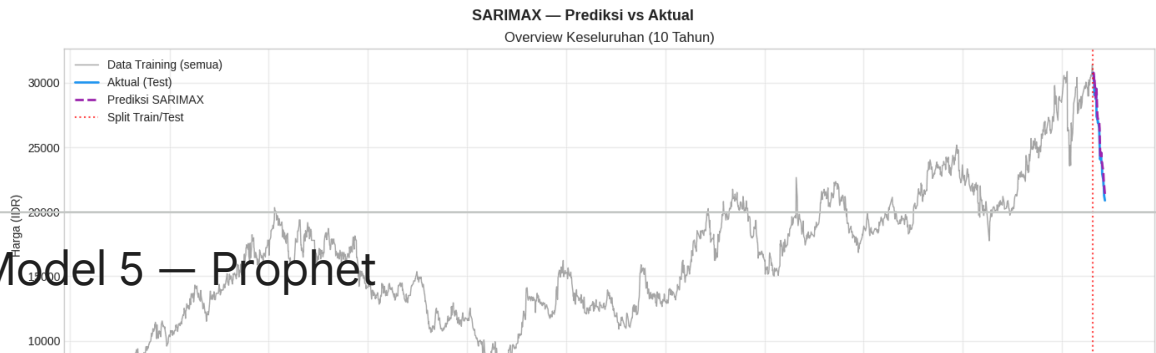
MAPE : 2.36%

(608.888726495236,

np.float64(741.9459683074298),

np.float64(2.364404466060673))


```
Fitting SARIMAX (Volume) + Walk-Forward...
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:4:
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:4:
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/base/model.py:607: Conv
warnings.warn("Maximum Likelihood optimization failed to ")
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:8:
return get_prediction_index(
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:8:
return get_prediction_index(
Selesai: 2.9s
```

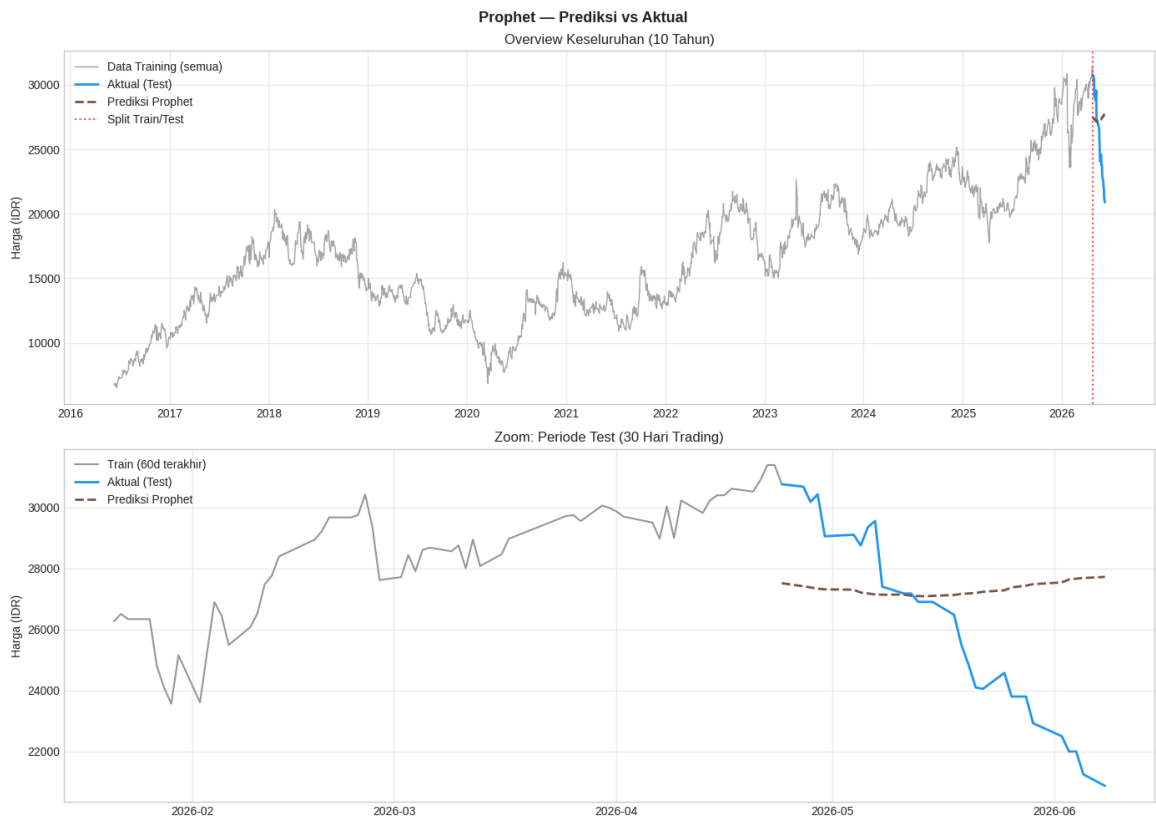


✓ 8. Model 5 — Prophet

```
prophet_df = pd.DataFrame({'ds': train_close.index, 'y': train_close.values.
m = Prophet(daily_seasonality=False, weekly_seasonality=True, yearly_seasona
m.fit(prophet_df)
future = m.make_future_dataframe(periods=n_test, freq='B')
fc = m.predict(future)
prophet_fc = fc.set_index('ds')['yhat']
prophet_fc = prophet_fc[prophet_fc.index >= test_close.index[0]].values[:n_t
plot_forecast('Prophet', prophet_fc, COLORS['Prophet'])
evaluate(test_close.values[:len(prophet_fc)], prophet_fc, 'Prophet')
```

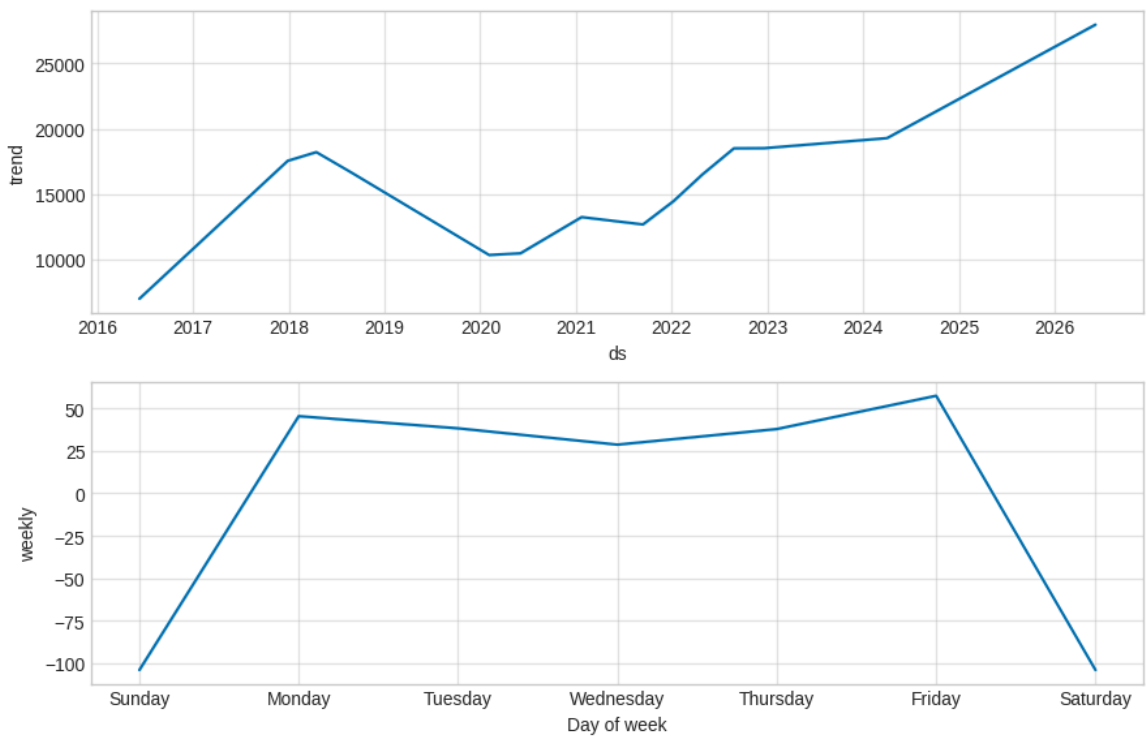
```
fig2 = m.plot_components(m.predict(future))
plt.suptitle('Prophet Components', y=1.02); plt.tight_layout(); plt.show()
```

```
Model : SARIMAX(Volume)
MAE : 487.67
RMSE : 660.50
MAPE : 1.92%
(487.667002300456,
 np.float64(660.5043693475967),
 np.float64(1.9166082472733652))
```

Model : Prophet
 MAE : 2,720.08
 RMSE : 3,319.77
 MAPE : 11.16%

Prophet Components



9. Model 6 — XGBoost


```

def make_features(series, lags=30):
    df_f = pd.DataFrame({'y': series})
    for i in range(1, lags+1): df_f[f'lag_{i}'] = df_f['y'].shift(i)
    df_f['rm7'] = df_f['y'].shift(1).rolling(7).mean()
    df_f['rm30'] = df_f['y'].shift(1).rolling(30).mean()
    df_f['rs7'] = df_f['y'].shift(1).rolling(7).std()
    df_f['dow'] = pd.DatetimeIndex(series.index).dayofweek
    df_f['month'] = pd.DatetimeIndex(series.index).month
    df_f['q'] = pd.DatetimeIndex(series.index).quarter
    return df_f.dropna()

feat = make_features(close)
fcols = [c for c in feat.columns if c != 'y']
is_train = feat.index <= train_close.index[-1]
X_tr, y_tr = feat.loc[is_train, fcols], feat.loc[is_train, 'y']
X_te, y_te = feat.loc[~is_train, fcols], feat.loc[~is_train, 'y']
xgb = XGBRegressor(n_estimators=500, learning_rate=0.05, max_depth=6,
                   subsample=0.8, colsample_bytree=0.8, random_state=42, ver
xgb.fit(X_tr, y_tr, eval_set=[(X_te, y_te)], verbose=False)
xgb_pred = xgb.predict(X_te)[:n_test]
plot_forecast('XGBoost', xgb_pred, COLORS['XGBoost'])
evaluate(test_close.values[:len(xgb_pred)], xgb_pred, 'XGBoost')

imp = pd.Series(xgb.feature_importances_, index=fcols).sort_values(ascending
fig, ax = plt.subplots(figsize=(8,6)); imp.plot.barh(ax=ax, color='#E91E63',
ax.set_title('Top 15 Feature Importance (XGBoost)'); plt.tight_layout(); plt

```

